

Li_Eddie_FinalReport

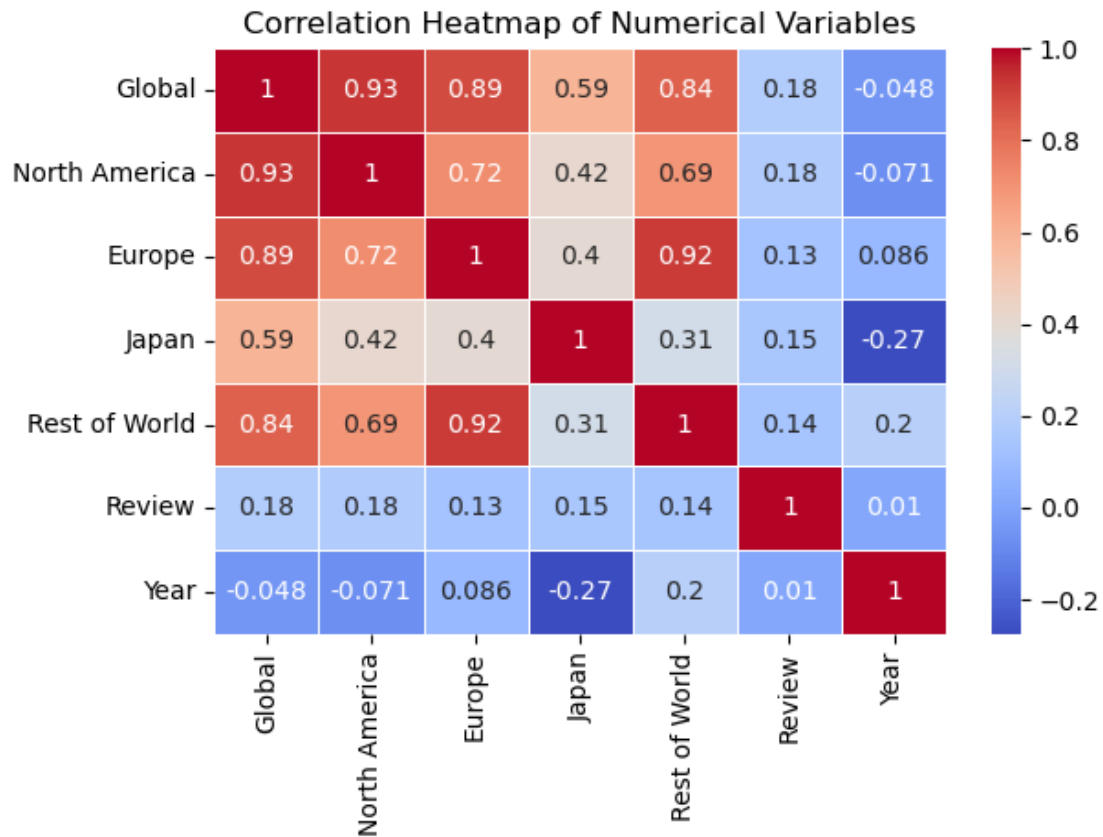
April 12, 2026

```
[1]: #Dataset setup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.model_selection import train_test_split
df = pd.read_csv("video_game_sales.csv")

[2]: #Identify numerical columns for correlation matrix
numerical_col = ["Global", "North America", "Europe", "Japan", "Rest of_
↪World", "Review", "Year"]

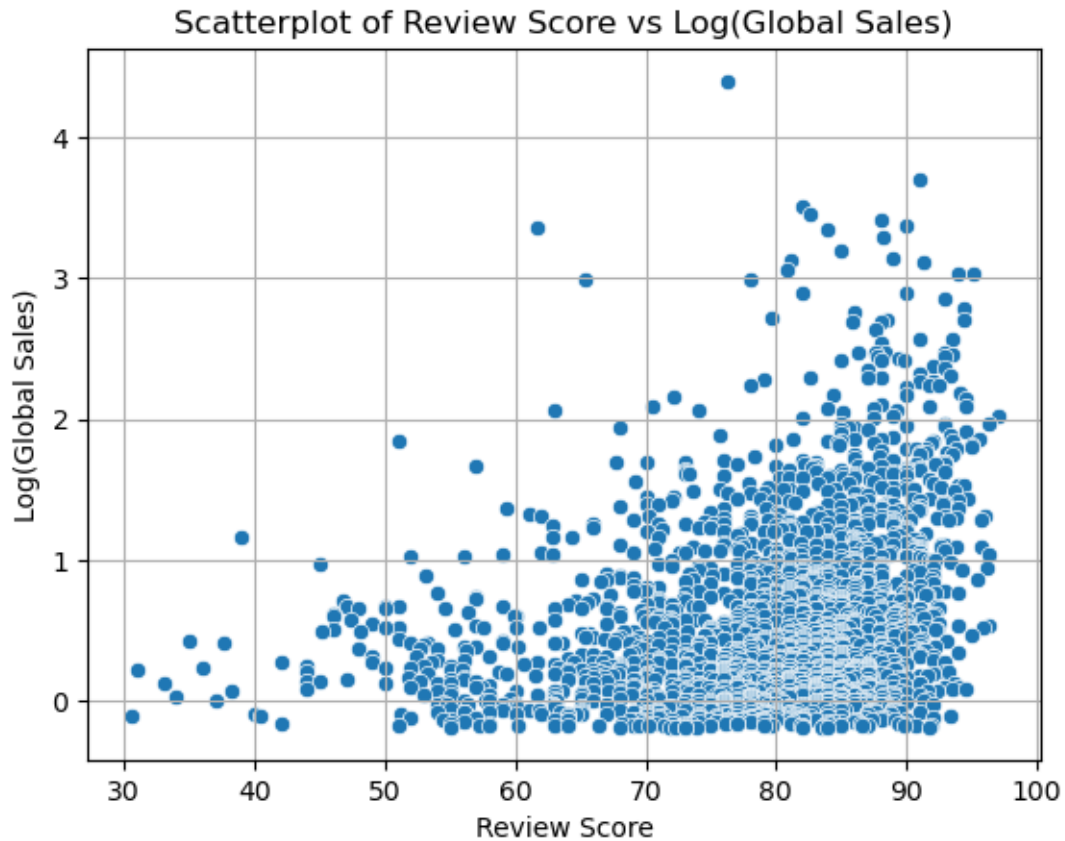
#Create correlation matrix heatmap
correlation_matrix = df[numerical_col].corr()
plt.figure() #Plot correlation matrix
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.5)

#Label the data visualization
plt.title("Correlation Heatmap of Numerical Variables")
plt.tight_layout()
plt.show()
```



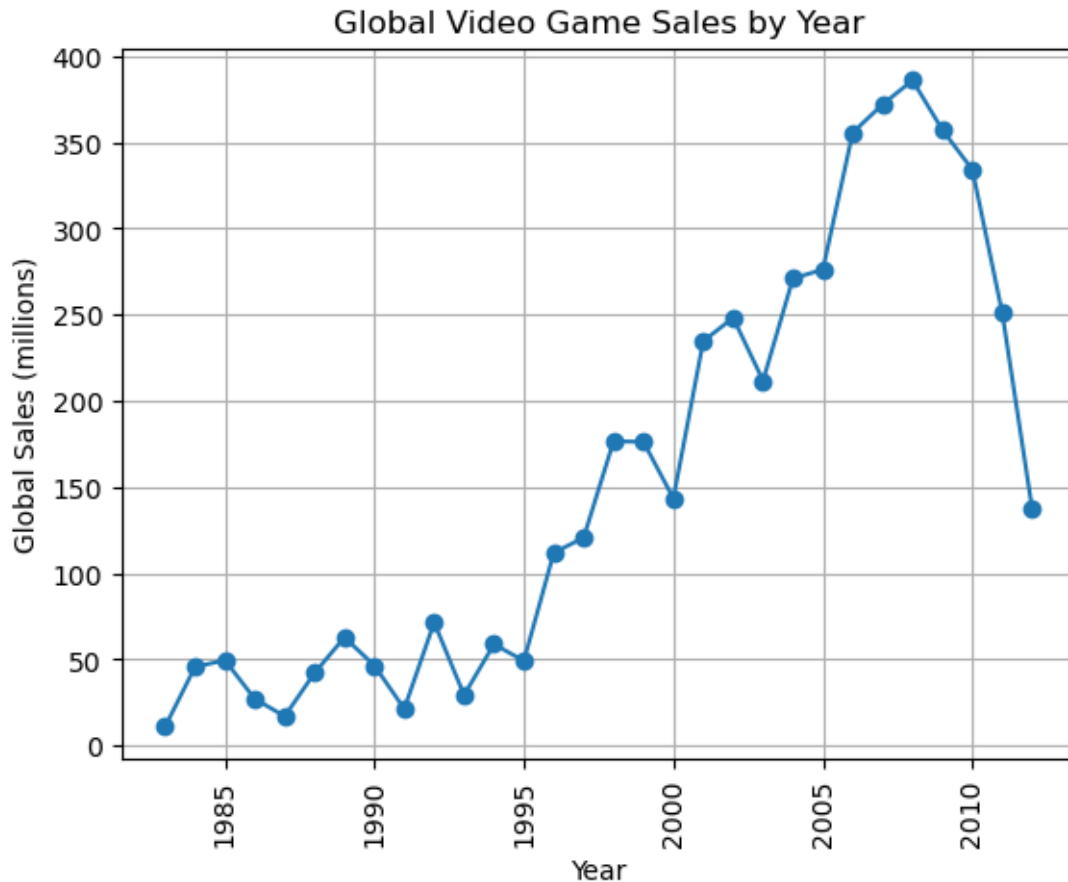
```
[3]: #Create scatter plot with Review and Log(Global Sales)
plt.figure()
sns.scatterplot(x="Review", y=np.log(df[df["Global"] > 0]["Global"]),
               data=df[df["Global"] > 0])

#Label data visualization
plt.title("Scatterplot of Review Score vs Log(Global Sales)")
plt.xlabel("Review Score")
plt.ylabel("Log(Global Sales)")
plt.grid(True)
plt.show()
```



```
[4]: #Create line graph for global sales and year
sales_by_year = df.groupby("Year")["Global"].sum().reset_index()
plt.figure()
plt.plot(sales_by_year["Year"], sales_by_year["Global"], marker='o')

#Label data visualization
plt.title("Global Video Game Sales by Year")
plt.xlabel("Year")
plt.ylabel("Global Sales (millions)")
plt.grid(True)
plt.xticks(rotation=90)
plt.show()
```



```
[5]: #Creates histograms for (log of) global and regional sales, if there are no
      ↪ sales it is assumed that the game did not release in that region and is not
      ↪ considered an outlier.
      #Regions are global, NA, EU, JP, and Rest of the World Sales.

      # Global Sales
      plt.figure()
      plt.hist(np.log(df[df["Global"] > 0]["Global"]), bins=30)
      plt.title("Histogram of Global Sales")
      plt.xlabel("Log(Global Sales)")
      plt.ylabel("Frequency")
      plt.show()

      # North America Sales
      plt.figure()
      plt.hist(np.log(df[df["North America"] > 0]["North America"]), bins=30)
      plt.title("Histogram of North American Sales")
      plt.xlabel("Log(North American Sales)")
```

```

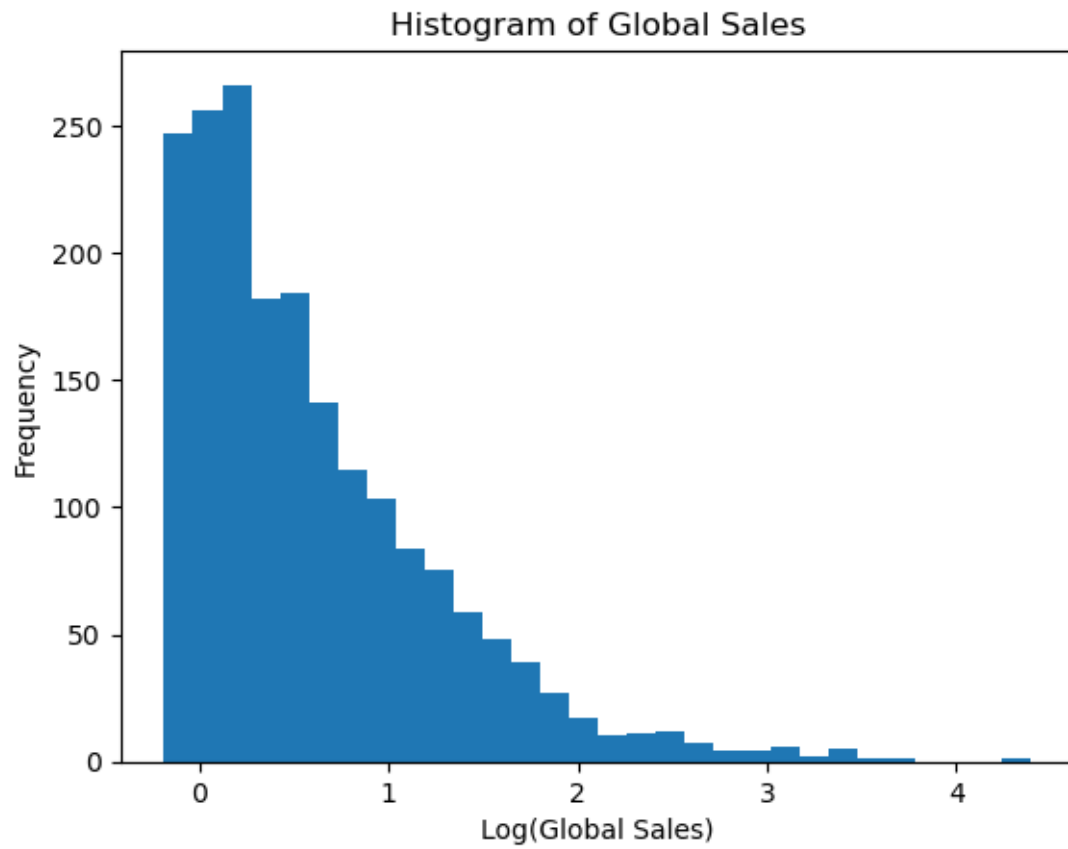
plt.ylabel("Frequency")
plt.show()

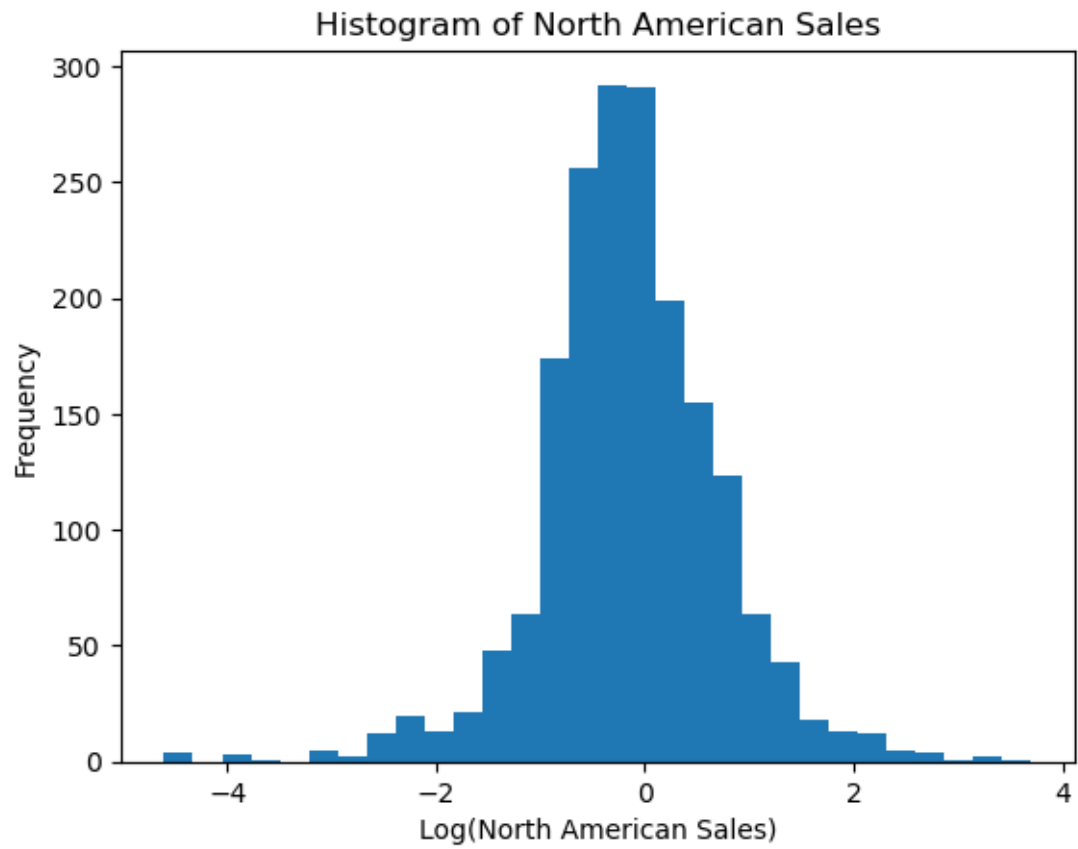
# European Sales
plt.figure()
plt.hist(np.log(df[df["Europe"] > 0]["Europe"]), bins=30)
plt.title("Histogram of European Sales")
plt.xlabel("Log(European Sales)")
plt.ylabel("Frequency")
plt.show()

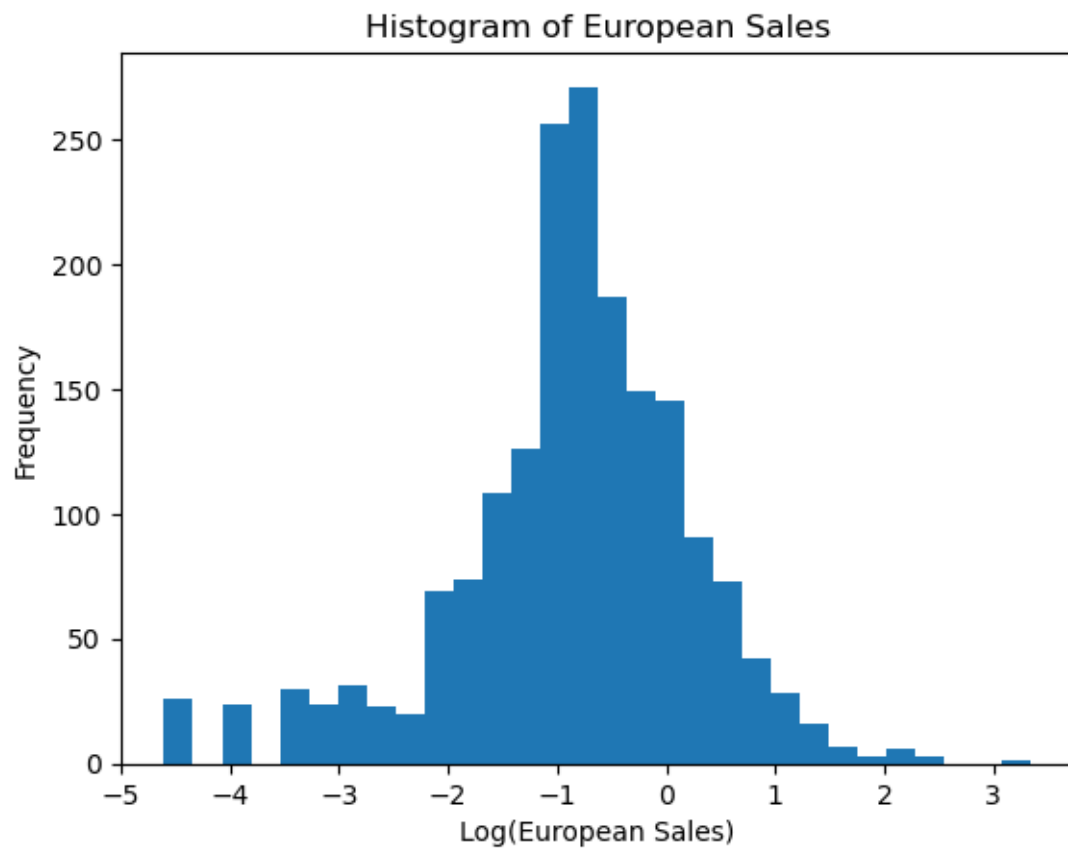
# Japan Sales
plt.figure()
plt.hist(np.log(df[df["Japan"] > 0]["Japan"]), bins=30)
plt.title("Histogram of Japan Sales (Non-Zero)")
plt.xlabel("Log(Japan Sales)")
plt.ylabel("Frequency")
plt.show()

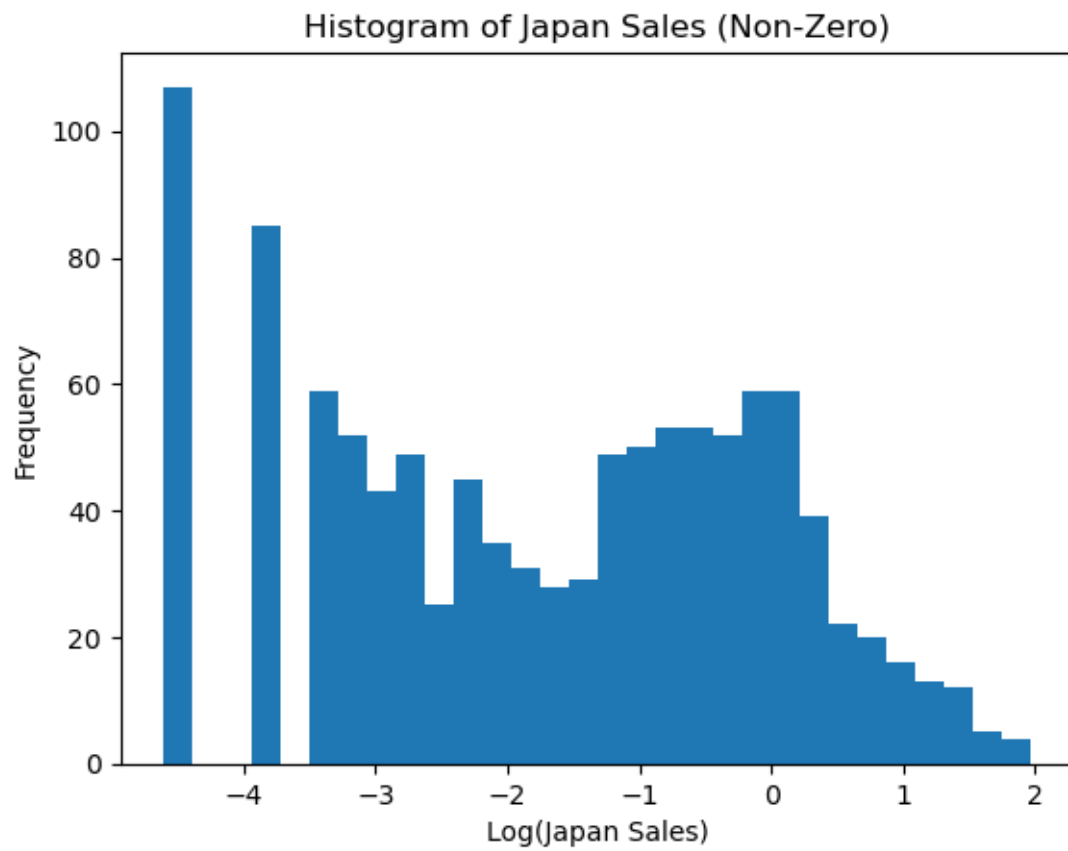
# Rest of World Sales
plt.figure()
plt.hist(np.log(df[df["Rest of World"] > 0]["Rest of World"]), bins=30)
plt.title("Histogram of Rest of World Sales")
plt.xlabel("Log(Rest of World Sales)")
plt.ylabel("Frequency")
plt.show()

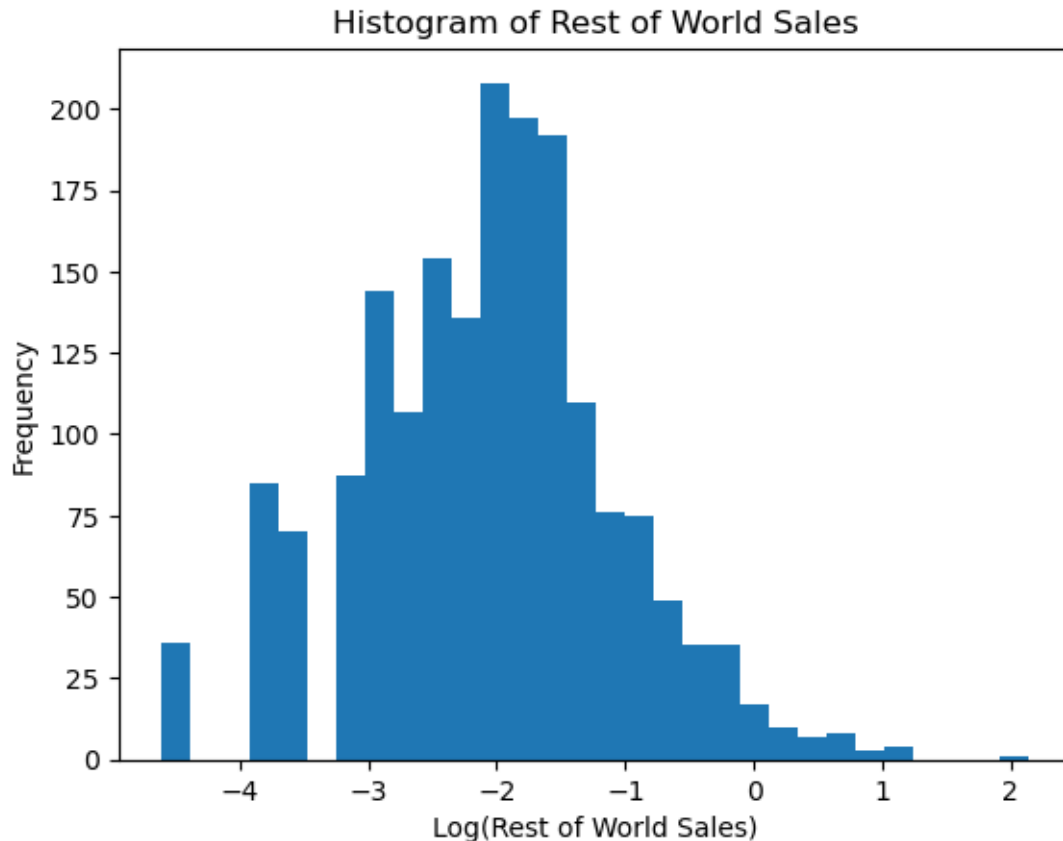
```











```
[6]: #Makes boxplots of global and regional sales, if there are no sales it is
      ↳ assumed that the game did not release in that region and is not considered
      ↳ an outlier.
      #Regions are global, NA, EU, JP, and Rest of the World Sales.
      #Visualizations are sorted from greatest to least median sales to identify and
      ↳ trends.

      #Global Sales
      genre_order = df[df["Global"] > 0].groupby("Genre")["Global"].median().
        ↳ sort_values(ascending=False).index
      plt.figure()
      sns.boxplot(x="Genre", y=np.log(df[df["Global"] > 0]["Global"]),
        ↳ data=df[df["Global"] > 0], order=genre_order)
      plt.title("Boxplot of Global Sales by Genre (Median Sales)")
      plt.xlabel("Genre")
      plt.ylabel("Log(Global Sales)")
      plt.xticks(rotation=90)
      plt.show()
```

```

# North America Sales
genre_order = df[df["North America"] > 0].groupby("Genre")["North America"].
    ↪median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Genre", y=np.log(df[df["North America"] > 0]["North America"]),
    ↪data=df[df["North America"] > 0], order=genre_order)
plt.title("Boxplot of North American Sales by Genre (Median Sales)")
plt.xlabel("Genre")
plt.ylabel("Log(North American Sales)")
plt.xticks(rotation=90)
plt.show()

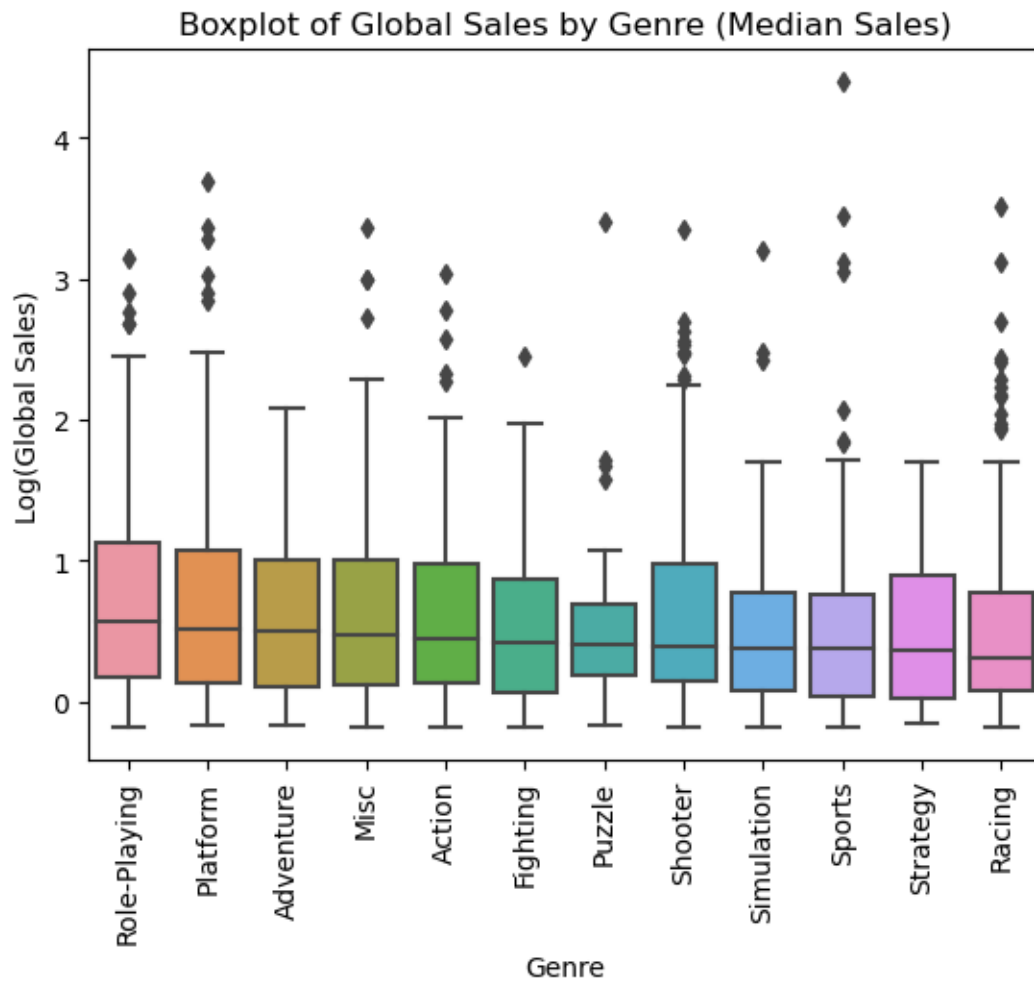
# Europe Sales
genre_order = df[df["Europe"] > 0].groupby("Genre")["Europe"].median().
    ↪sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Genre", y=np.log(df[df["Europe"] > 0]["Europe"]),
    ↪data=df[df["Europe"] > 0], order=genre_order)
plt.title("Boxplot of European Sales by Genre (Median Sales)")
plt.xlabel("Genre")
plt.ylabel("Log(European Sales)")
plt.xticks(rotation=90)
plt.show()

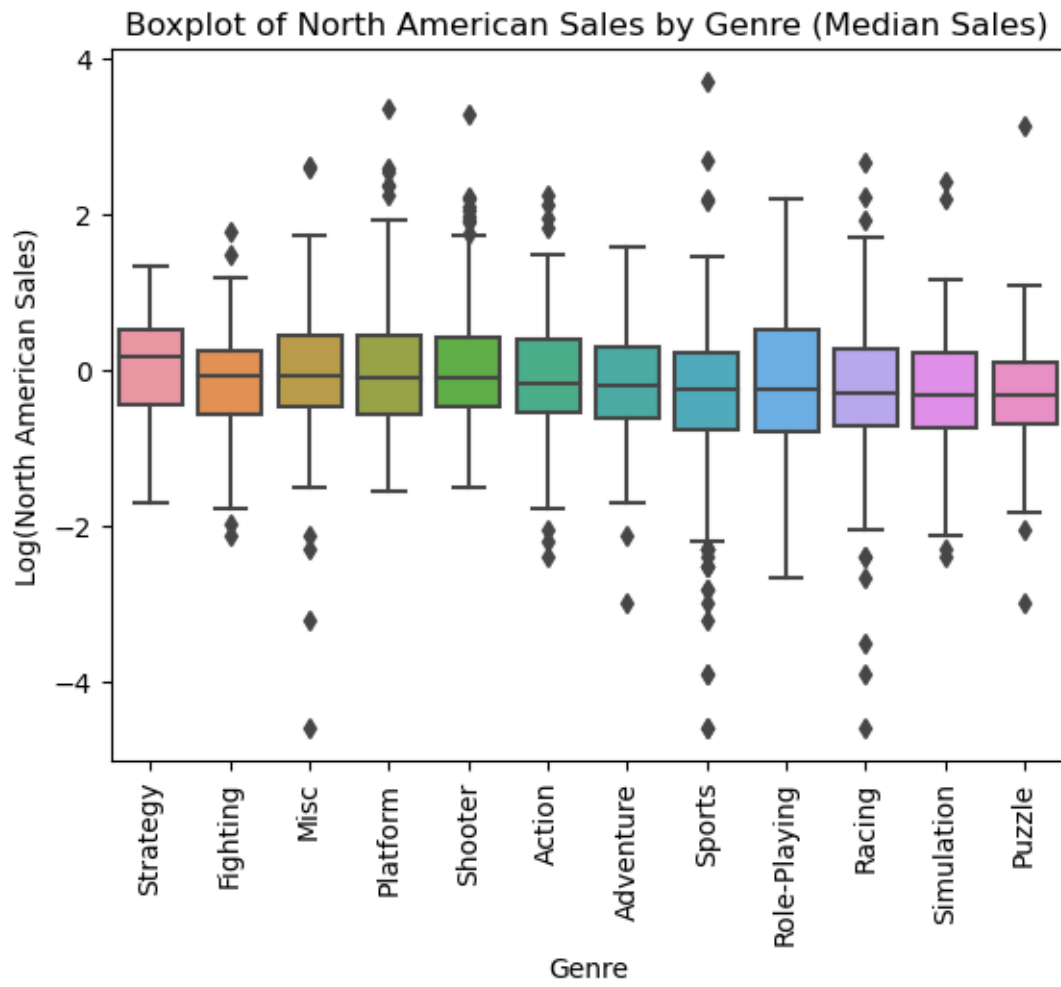
# Japan Sales
genre_order = df[df["Japan"] > 0].groupby("Genre")["Japan"].median().
    ↪sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Genre", y=np.log(df[df["Japan"] > 0]["Japan"]),
    ↪data=df[df["Japan"] > 0], order=genre_order)
plt.title("Boxplot of Japan Sales by Genre (Median Sales)")
plt.xlabel("Genre")
plt.ylabel("Log(Japan Sales)")
plt.xticks(rotation=90)
plt.show()

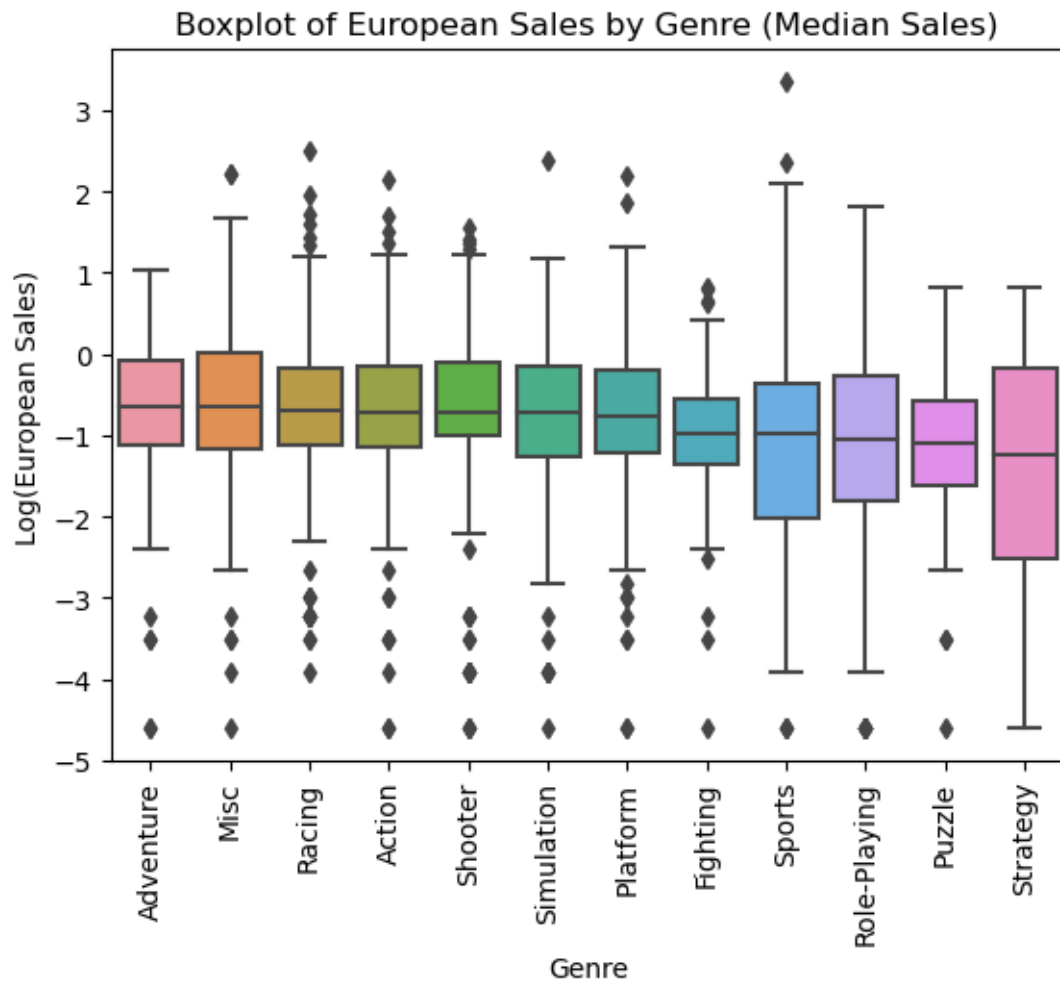
# Rest of World Sales
genre_order = df[df["Rest of World"] > 0].groupby("Genre")["Rest of World"].
    ↪median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Genre", y=np.log(df[df["Rest of World"] > 0]["Rest of World"]),
    ↪data=df[df["Rest of World"] > 0], order=genre_order)
plt.title("Boxplot of Rest of World Sales by Genre (Median Sales)")
plt.xlabel("Genre")
plt.ylabel("Log(Rest of World Sales)")
plt.xticks(rotation=90)

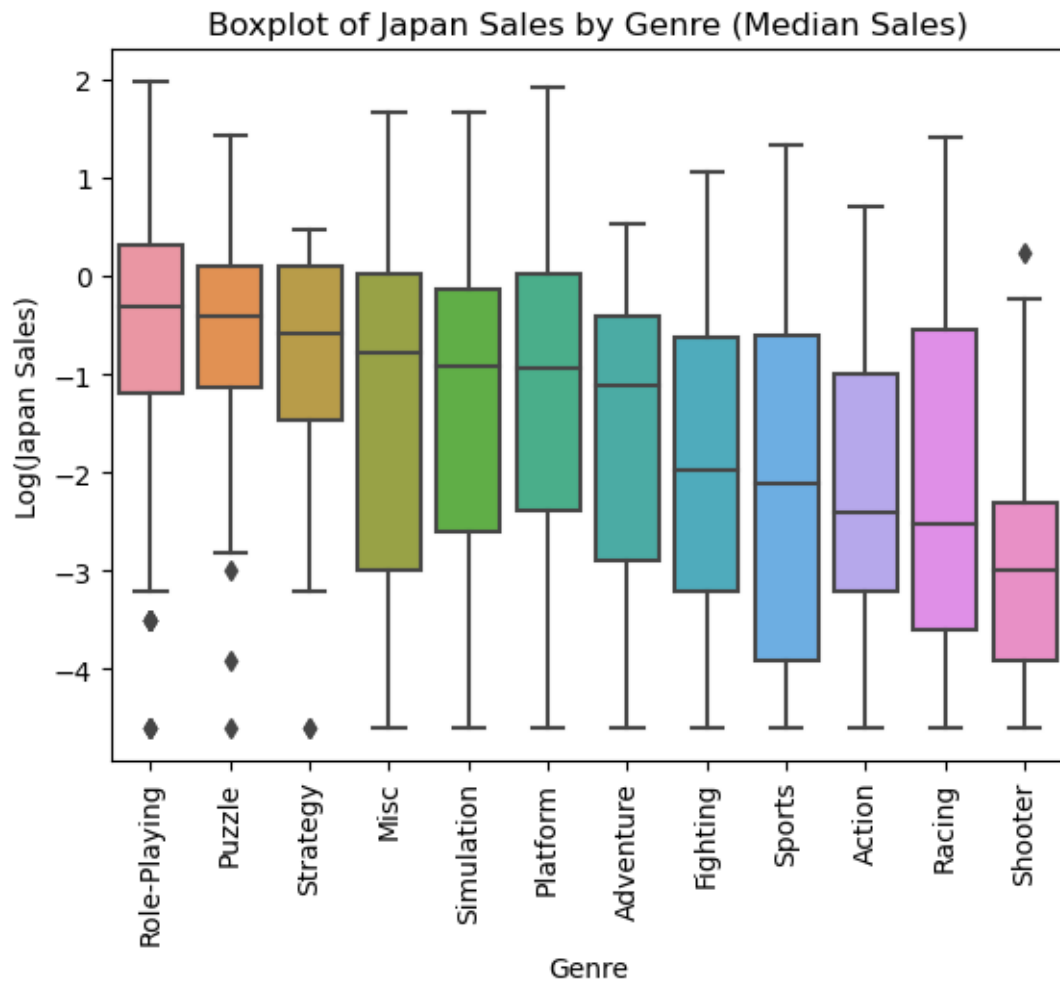
```

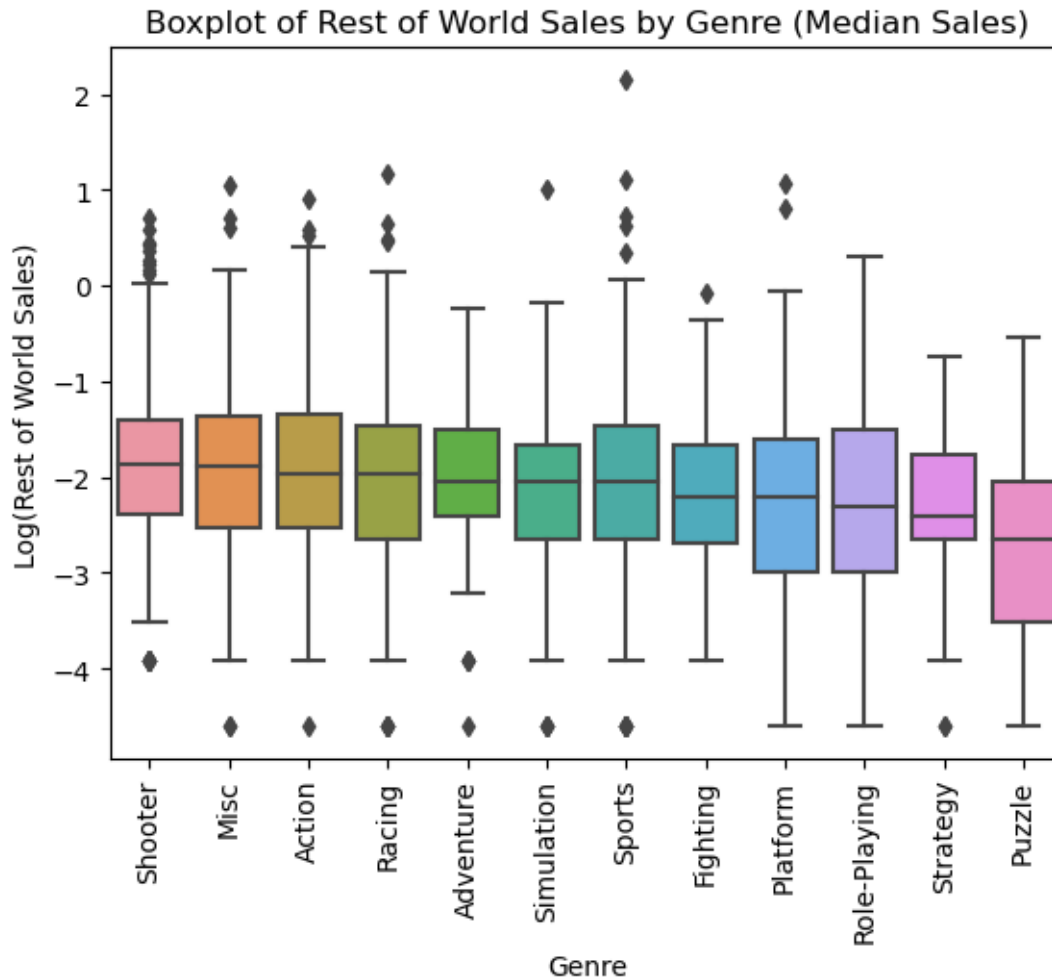
```
plt.show()
```











```
[7]: #Makes boxplots of global and regional sales but for platform instead of genre,
      ↳ if there are no sales it is assumed that the game did not release in that
      ↳ region and is not considered an outlier.
      #Regions are global, NA, EU, JP, and Rest of the World Sales.
      #Visualizations are sorted from greatest to least median sales to identify and
      ↳ trends.

      # Global Sales by Platform
      platform_order = df[df["Global"] > 0].groupby("Platform")["Global"].median().
        ↳ sort_values(ascending=False).index
      plt.figure()
      sns.boxplot(x="Platform", y=np.log(df[df["Global"] > 0]["Global"]),
        ↳ data=df[df["Global"] > 0], order=platform_order)
      plt.title("Boxplot of Global Sales by Platform (Median Sales)")
      plt.xlabel("Platform")
      plt.ylabel("Log(Global Sales)")
```



```

plt.xticks(rotation=90)
plt.show()

# North America Sales by Platform
platform_order = df[df["North America"] > 0].groupby("Platform")["North_America"].median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Platform", y=np.log(df[df["North America"] > 0]["North_America"]), data=df[df["North America"] > 0], order=platform_order)
plt.title("Boxplot of North American Sales by Platform (Median Sales)")
plt.xlabel("Platform")
plt.ylabel("Log(North American Sales)")
plt.xticks(rotation=90)
plt.show()

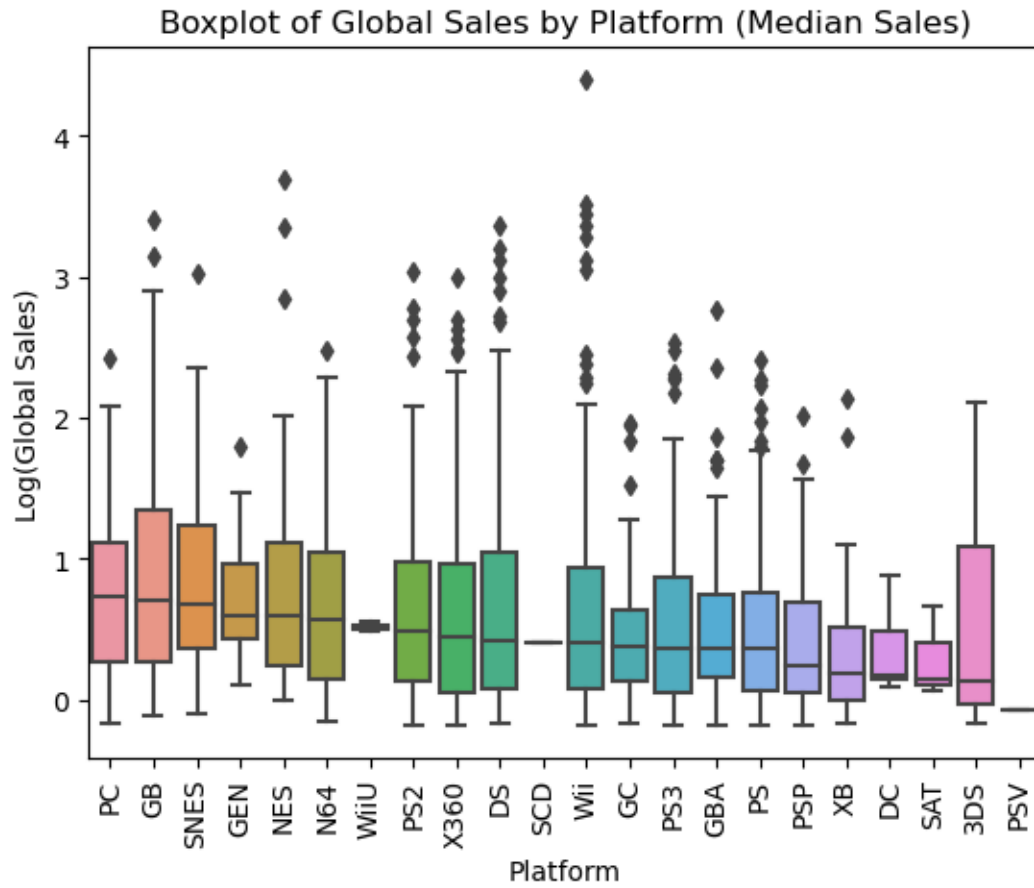
# Europe Sales by Platform
platform_order = df[df["Europe"] > 0].groupby("Platform")["Europe"].median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Platform", y=np.log(df[df["Europe"] > 0]["Europe"]), data=df[df["Europe"] > 0], order=platform_order)
plt.title("Boxplot of European Sales by Platform (Median Sales)")
plt.xlabel("Platform")
plt.ylabel("Log(European Sales)")
plt.xticks(rotation=90)
plt.show()

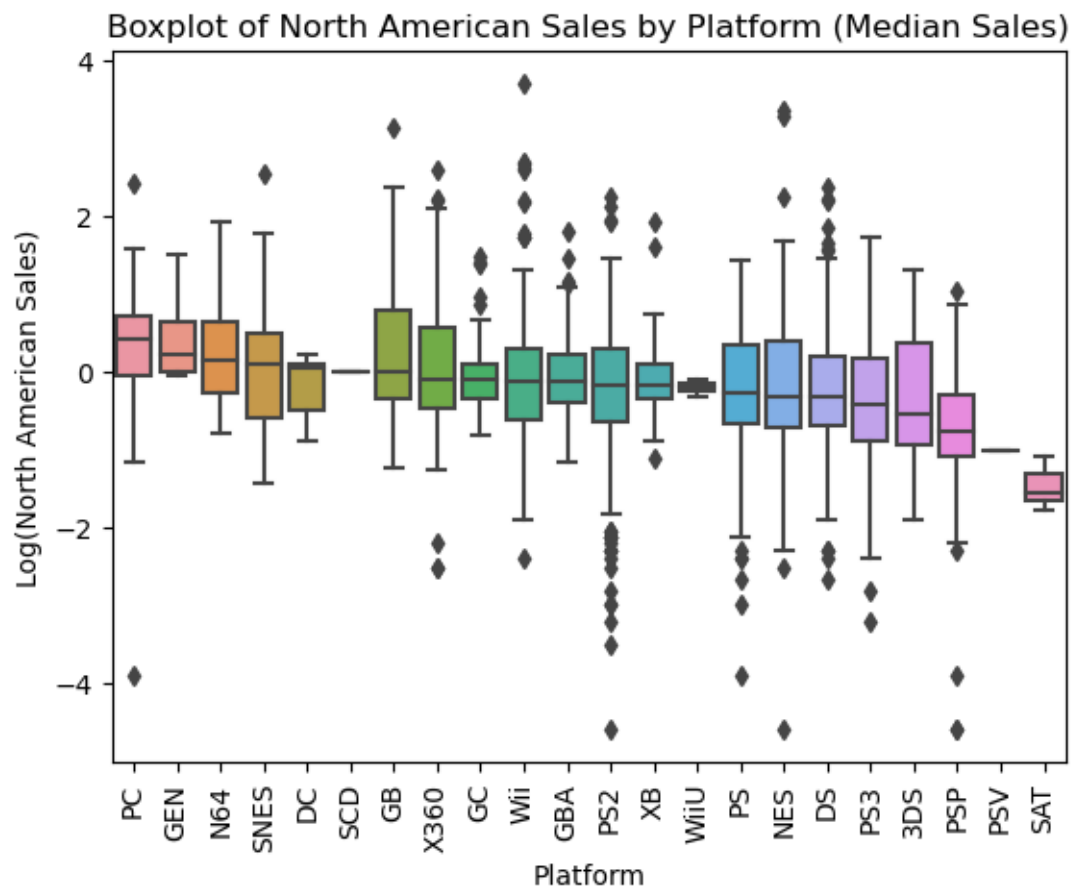
# Japan Sales by Platform
platform_order = df[df["Japan"] > 0].groupby("Platform")["Japan"].median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Platform", y=np.log(df[df["Japan"] > 0]["Japan"]), data=df[df["Japan"] > 0], order=platform_order)
plt.title("Boxplot of Japan Sales by Platform (Median Sales)")
plt.xlabel("Platform")
plt.ylabel("Log(Japan Sales)")
plt.xticks(rotation=90)
plt.show()

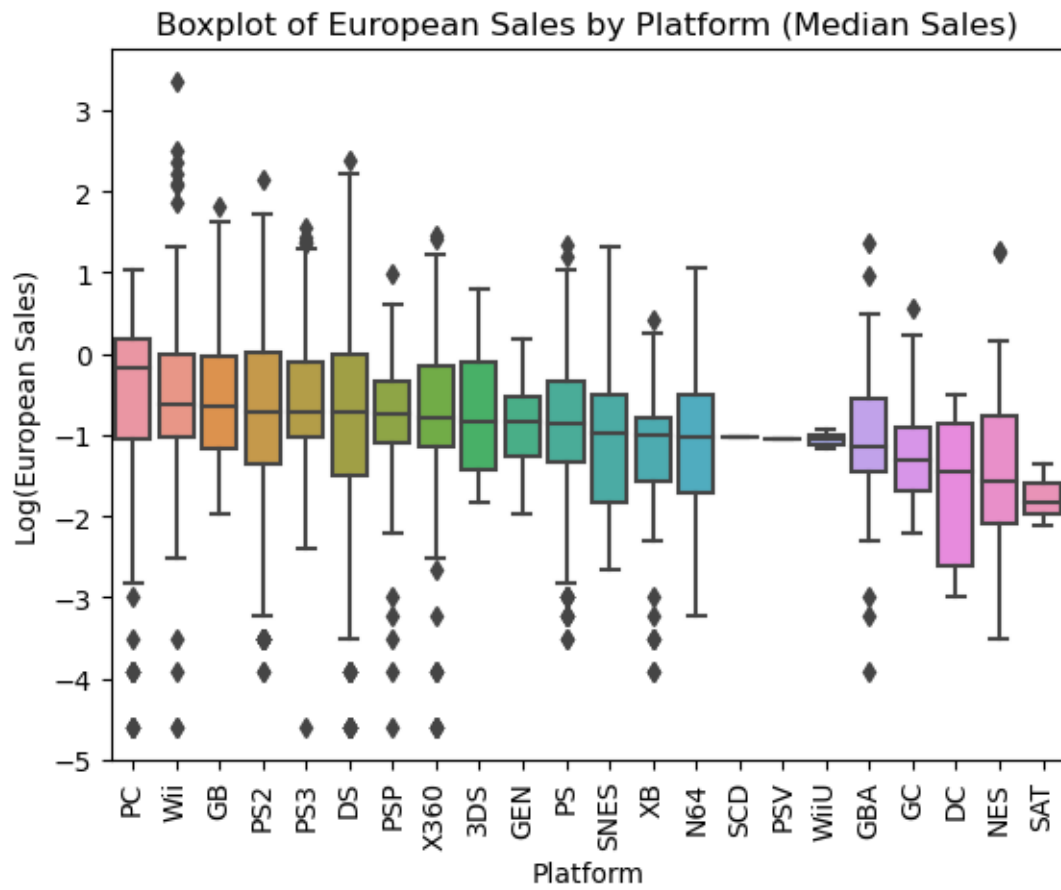
# Rest of World Sales by Platform
platform_order = df[df["Rest of World"] > 0].groupby("Platform")["Rest of_World"].median().sort_values(ascending=False).index
plt.figure()
sns.boxplot(x="Platform", y=np.log(df[df["Rest of World"] > 0]["Rest of_World"]), data=df[df["Rest of World"] > 0], order=platform_order)
plt.title("Boxplot of Rest of World Sales by Platform (Median Sales)")

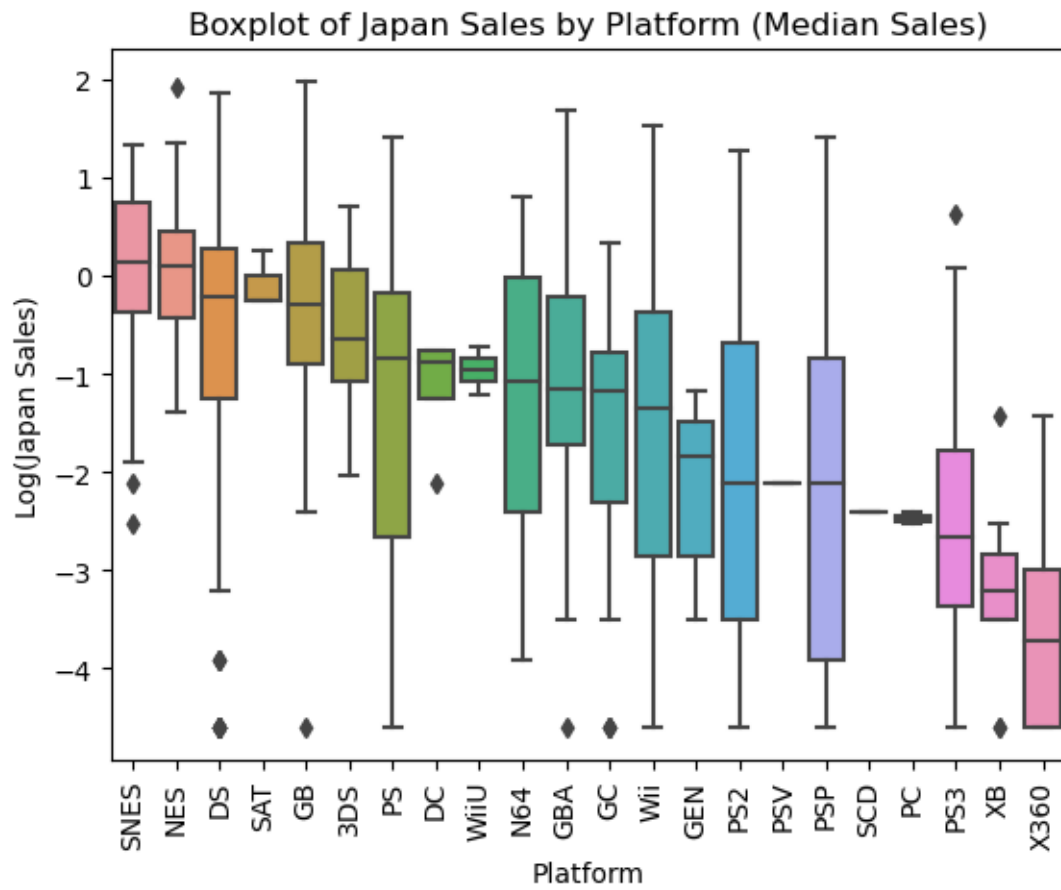
```

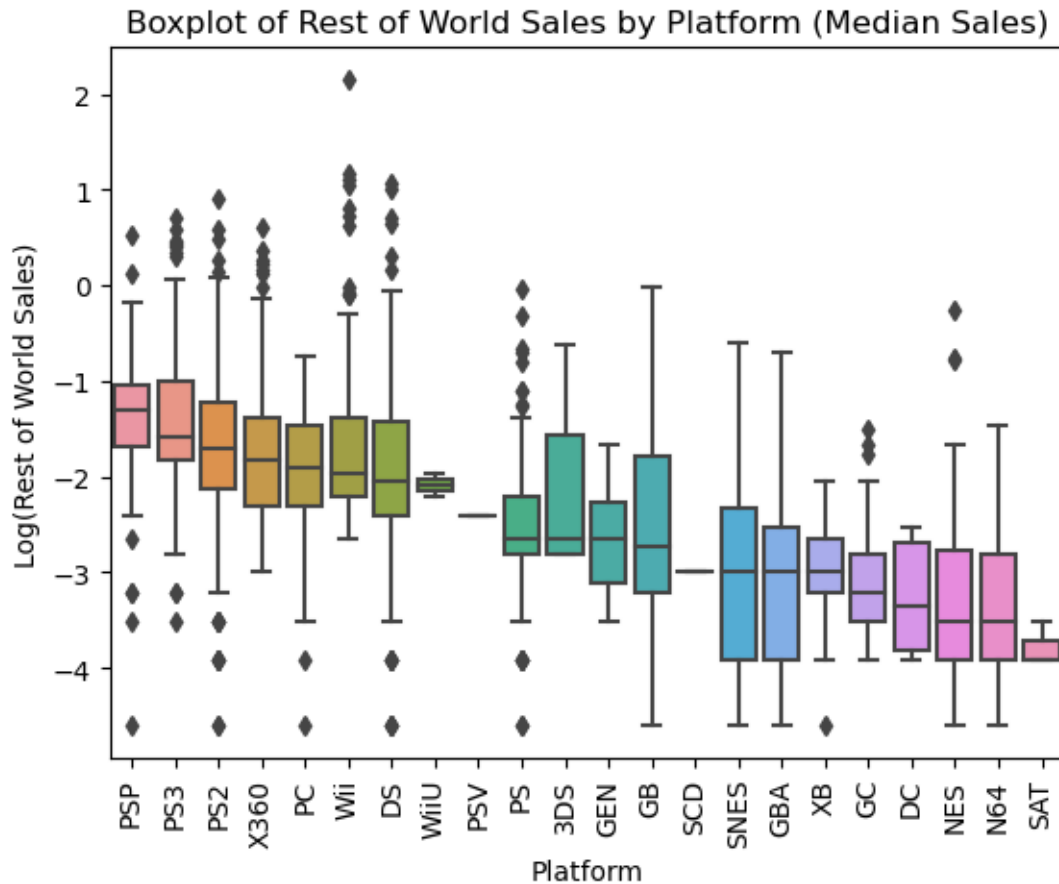
```
plt.xlabel("Platform")
plt.ylabel("Log(Rest of World Sales)")
plt.xticks(rotation=90)
plt.show()
```











```
[8]: #Drop empty year entries and reassign datatypes
df = df.dropna(subset=['Year'])
df['Year'] = df['Year'].astype('int64')
object_col = df.select_dtypes(include=['object']).columns
df[object_col] = df[object_col].astype('string')

#Round float data types to 2 decimal places
float_col = df.select_dtypes(include=['float64']).columns
df[float_col] = df[float_col].round(2)
```

```
[19]: #Machine learning models setup
from sklearn.model_selection import train_test_split, KFold, cross_val_score, \
    cross_val_predict, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.neural_network import MLPRegressor
from sklearn.svm import SVR
import time

#Select factor and target variables
X = df[["Platform", "Genre", "Review", "Year"]]
y = df["Global"]

#Convert categorical into dummy numerical variables
#split the data set into 80/20 and fix at random state of 67
X_encoded = pd.get_dummies(X, columns=["Platform", "Genre"], drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.
    ↪2, random_state=67)

#Scales the data based on the testing data input
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

```

[12]: #Random Forest Machine Learning Model
model = RandomForestRegressor(n_estimators=100, random_state=67)

#Trains the current machine learning model using the training data and creates
    ↪predictions using the model
#Also measures time taken to train the model and time taken to make prediction
start_train = time.time()
model.fit(X_train, y_train)
end_train = time.time()
training_time = end_train - start_train

start_pred = time.time()
y_pred = model.predict(X_test)
end_pred = time.time()
prediction_time = end_pred - start_pred

#Calculates coefficient of determination, root mean squared error, and mean
    ↪absolute error
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

#Results of the current machine learning model
print("R²:", r2)
print("RMSE:", rmse)
print("MAE:", mae)
print("Training Time:", training_time)
print("Prediction Time:", prediction_time)

```

R²: 0.18405623060151488
RMSE: 3.072901073073085
MAE: 1.5967288500506585
Training Time: 0.5729622840881348
Prediction Time: 0.014252424240112305

```
[17]: #Gradient Boosting machine learning model
model = GradientBoostingRegressor(n_estimators=200, random_state=67)

#Trains the current machine learning model using the training data and creates
↳ predictions using the model
#Also measures time taken to train the model and time taken to make prediction
start_train = time.time()
model.fit(X_train, y_train)
end_train = time.time()
training_time = end_train - start_train

start_pred = time.time()
y_pred = model.predict(X_test)
end_pred = time.time()
prediction_time = end_pred - start_pred

#Calculates coefficient of determination, root mean squared error, and mean
↳ absolute error
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

#Results of the current machine learning model
print("R2:", r2)
print("RMSE:", rmse)
print("MAE:", mae)
print("Training Time:", training_time)
print("Prediction Time:", prediction_time)
```

R²: 0.10028809248990023
RMSE: 3.2267862527016016
MAE: 1.6365939007752621
Training Time: 0.21860146522521973
Prediction Time: 0.0010001659393310547

```
[13]: #K-Nearest Neighbour machine learning model
model = KNeighborsRegressor(n_neighbors=10)

#Trains the current machine learning model using the training data and creates
↳ predictions using the model
#Also measures time taken to train the model and time taken to make prediction
start_train = time.time()
```



```

model.fit(X_train, y_train)
end_train = time.time()
training_time = end_train - start_train

start_pred = time.time()
y_pred = model.predict(X_test)
end_pred = time.time()
prediction_time = end_pred - start_pred

#Calculates coefficient of determination, root mean squared error, and mean
↪absolute error
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

#Results of the current machine learning model
print("R²:", r2)
print("RMSE:", rmse)
print("MAE:", mae)
print("Training Time:", training_time)
print("Prediction Time:", prediction_time)

```

```

R²: 0.10714696199741836
RMSE: 3.214463170329418
MAE: 1.6477127659574466
Training Time: 0.0010008811950683594
Prediction Time: 0.1816861629486084

```

```

[14]: #Artificial Neural Network machine learning model
model = MLPRegressor(hidden_layer_sizes=(100,50), max_iter=500, random_state=67)

#Trains the current machine learning model using the training data and creates
↪predictions using the model
#Also measures time taken to train the model and time taken to make prediction
start_train = time.time()
model.fit(X_train, y_train)
end_train = time.time()
training_time = end_train - start_train

start_pred = time.time()
y_pred = model.predict(X_test)
end_pred = time.time()
prediction_time = end_pred - start_pred

#Calculates coefficient of determination, root mean squared error, and mean
↪absolute error
r2 = r2_score(y_test, y_pred)

```

```

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

#Results of the current machine learning model
print("R²:", r2)
print("RMSE:", rmse)
print("MAE:", mae)
print("Training Time:", training_time)
print("Prediction Time:", prediction_time)

```

R²: 0.21576135076443315
RMSE: 3.012607719979547
MAE: 1.660465854214327
Training Time: 1.6030380725860596
Prediction Time: 0.0

```

[15]: #Support Vector Regression machine learning model
model = SVR(kernel="rbf") #radical basis function, for non-linear functions

#Trains the current machine learning model using the training data and creates
↳ predictions using the model
#Also measures time taken to train the model and time taken to make prediction
start_train = time.time()
model.fit(X_train, y_train)
end_train = time.time()
training_time = end_train - start_train

start_pred = time.time()
y_pred = model.predict(X_test)
end_pred = time.time()
prediction_time = end_pred - start_pred

#Calculates coefficient of determination, root mean squared error, and mean
↳ absolute error
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

#Results of the current machine learning model
print("R²:", r2)
print("RMSE:", rmse)
print("MAE:", mae)
print("Training Time:", training_time)
print("Prediction Time:", prediction_time)

```

R²: -0.010932082181333902
RMSE: 3.420420128158757
MAE: 1.3776506077648982

Training Time: 0.07716846466064453
Prediction Time: 0.030007362365722656

```
[20]: #5-fold cross validation

#Splits the data set into 5 random equal parts
cv = KFold(n_splits=5, shuffle=True, random_state=67)

#Models
models = {
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=67),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=200,
    random_state=67),
    "KNN": KNeighborsRegressor(n_neighbors=10),
    "ANN": MLPRegressor(hidden_layer_sizes=(100,50), max_iter=500,
    random_state=67),
    "SVR": SVR(kernel="rbf")
}

scoring = {
    "r2": "r2",
    "rmse": "neg_root_mean_squared_error",
    "mae": "neg_mean_absolute_error"
}

#Calculate and print average values for the 5-fold cross validation
for name, model in models.items():

    #Scales the data for each machine learning model
    pipeline = Pipeline([("scaler", StandardScaler()),("model", model)])

    start_time = time.time()

    #Calculates the average coefficient of determination, root mean squared
    error, and mean absolute error for each machine learning model
    scores = cross_validate(pipeline,X_encoded,y,cv=cv,scoring=scoring)

    end_time = time.time()
    total_time = end_time - start_time

    r2 = scores["test_r2"].mean()
    rmse = abs(scores["test_rmse"].mean())
    mae = abs(scores["test_mae"].mean())

    #Prints the results for each machine learning model
    print(name)
    print("R2:", r2)
```

```
print("RMSE:", rmse)
print("MAE:", mae)
print("Cross-Validation Time:", total_time)
print()
```

Random Forest

R²: -0.0459356568078787
RMSE: 3.4397680428294763
MAE: 1.5422924795481256
Cross-Validation Time: 2.893303155899048

Gradient Boosting

R²: -0.08019447161467592
RMSE: 3.5099965175369987
MAE: 1.5553952863023848
Cross-Validation Time: 1.1061582565307617

KNN

R²: -0.03099910184610437
RMSE: 3.4792493853985524
MAE: 1.5874727773049646
Cross-Validation Time: 0.04400920867919922

ANN

R²: -0.23934160572197163
RMSE: 3.753962054303597
MAE: 1.7551057091997877
Cross-Validation Time: 8.00230073928833

SVR

R²: -0.009798642221858177
RMSE: 3.4746992945321793
MAE: 1.3176274302884015
Cross-Validation Time: 0.571441650390625

[]: